Benefits, Challenges and Security

Splitting Notes into a gateway, a data service and a database makes each part scalable and replaceable on its own. The cost is more moving parts, network calls between services and more places to secure.

Benefits

Benefit	How the design delivers it	Business implication
Independent scaling	`web` and `notes-api` are separate stateless Deployments; each is scaled with its own `kubectl scale`	Pay only for capacity where load is. Heavy page traffic needs more `web` pods, not a bigger database tier.
Failure isolation	`web` times out after 3 seconds and returns `503` if `notes-api` fails; Kubernetes restarts crashed pods	One faulty service degrades the app instead of taking it down. Problems are easier to contain.
High availability of the services	2 replicas each, readiness probes, Service load balancing	One pod can crash or be updated while the other keeps serving users.
Zero-downtime updates	Rolling updates per Deployment	New versions ship during business hours without a maintenance window.
Independent development	Each service has its own code, image and REST contract	Teams can change, test and release one service without coordinating a full release.
Smaller attack surface	Only `web` is reachable from outside; the database is internal	Fewer exposed entry points means lower risk of a data breach.
Portability	Plain Docker images and standard Kubernetes objects in one manifest	Runs on Docker Desktop, minikube, k3s or any cloud provider, with no vendor lock-in.
Durable data	MongoDB on a PersistentVolumeClaim	Notes survive pod, deployment and node restarts, so users don't lose work.

Challenges and mitigations

Challenge	Why it matters	What I have done	What can be done
Network calls between services	Every request takes an extra hop, adds latency and can fail half way	3-second timeout on every call from `web`; clear `503` instead of hanging; errors in `notes-api` return `500` instead of crashing	Retries with backoff for reads, a circuit breaker, caching the note list in `web`
Database is a single point of failure	One MongoDB pod: if it is down, nothing can be read or written	Data on a PersistentVolumeClaim; `Recreate` strategy so two pods never write the same volume; Kubernetes restarts the pod	MongoDB replica set in a StatefulSet, or a managed database such as MongoDB Atlas, plus scheduled backups
Database does not scale	Only the stateless tiers scale horizontally	Acknowledged: the database is allowed to be a single instance	Replica set for read scaling, sharding for write scaling, indexes as data grows
Health check is shallow	`notes-api` reports ready even if MongoDB is unreachable	`/health` endpoints and readiness probes on both services	Make readiness ping MongoDB; add liveness probes, and a probe and resource limits on `mongo`
Startup order	`notes-api` can start before MongoDB is ready	The MongoDB driver reconnects on its own and Kubernetes restarts failed pods	An init container that waits for MongoDB, or a readiness check that depends on it
Harder debugging and monitoring	A request crosses several pods and logs are spread out	Every request is logged to stdout (`kubectl logs`); `X-Served-By` headers show which pod answered	Central logging (Loki or ELK), metrics (Prometheus and Grafana), distributed tracing (OpenTelemetry)
Manual scaling	Someone has to notice the load and run `kubectl scale`	CPU and memory requests and limits are already set on both services	A HorizontalPodAutoscaler per Deployment, scaling on CPU

Challenge	Why it matters	What I have done	What can be done
More operational complexity	Three services, images and Kubernetes objects to build, version and deploy	Everything in one manifest; versioned image tags (`1.0`); lockfiles with `npm ci` for repeatable builds	A CI/CD pipeline that builds, tests, scans and deploys; Helm or Kustomize for per-environment settings
Keeping the API contract in sync	A change in `notes-api` can break `web`	Small, documented REST API with stable status codes	Versioned paths (`/v1/notes`), an OpenAPI specification, contract tests in CI

Security

The design keeps the database and internal API off the network edge and validates all input. Its biggest gaps are that users don't log in, traffic is not encrypted, and the demo Secret is committed to git in plain text.

Risk	What I have done	What can be done
Exposed attack surface	Only the `web` Service is a NodePort; `notes-api` and `mongo` are ClusterIP and cannot be reached from outside the cluster. On a cloud VM, a firewall allows only SSH and port 30080.	An Ingress controller as the single entry point, with a web application firewall
No authentication or authorization	Not done: any visitor can add or delete any note	User login with OAuth2 / OpenID Connect; `web` passes a token that `notes-api` checks; notes owned per user
Plain HTTP, no encryption	Not done: traffic between the browser and `web`, and between services, is unencrypted	TLS on the Ingress with cert-manager and Let's Encrypt; mutual TLS between services with a service mesh (Istio or Linkerd)

Risk	What I have done	What can be done
Database credentials in plain text	MongoDB requires a password. Credentials come from a Kubernetes Secret, not the code or image. **Weakness:** the demo Secret is committed in `k8s/app.yaml`, and base64 is not encryption.	Sealed Secrets or External Secrets with a cloud secret manager; etcd encryption at rest; RBAC limiting who can read Secrets
Application uses the database root account	Not done: `notes-api` connects as the MongoDB root user	A dedicated MongoDB user with read and write rights on `notesdb` only (least privilege)
Injection and cross-site scripting	`notes-api` checks text is 1–500 characters and ids are 24 hex characters; the MongoDB driver uses structured queries, not strings; the page shows notes with `textContent`, so HTML in a note is not run	A Content-Security-Policy header and other security headers (e.g. with `helmet`)
Information leakage	Errors return a generic `internal error`; details go only to the server log	Remove the `X-Served-By` pod-name headers outside of demos
Abuse and denial of service	CPU and memory limits on both services cap how much one pod can consume; Express limits JSON bodies to 100 KB by default	Rate limiting per client (in `web` or at the Ingress); autoscaling with an upper limit
Lateral movement inside the cluster	Not done: any pod in the cluster can call `notes-api` and `mongo`	NetworkPolicies allowing only `web` → `notes-api` and `notes-api` → `mongo`
Container and supply-chain risk	Small `node:20-alpine` base image; containers run as the non-root `node` user; dependencies pinned with lockfiles and installed with `npm ci`	Image scanning (Trivy) and `npm audit` in CI; read-only root filesystem and dropped Linux capabilities; signed images; pin `mongo:7` to an exact version

Next steps

If this application went to production, I would add these first, in order of risk:

1. **Encrypt traffic and add login:** an Ingress with TLS, and OAuth2 / OpenID Connect authentication.
2. **Protect the credentials:** move the Secret out of git (Sealed Secrets or a cloud secret manager) and give `notes-api` its own least-privilege MongoDB user.
3. **Isolate services:** NetworkPolicies so only `web` can call `notes-api` and only `notes-api` can reach `mongo`.
4. **Make the database resilient:** a MongoDB replica set or a managed database, with backups.
5. **Automate operations:** HorizontalPodAutoscalers, readiness checks that include MongoDB, central logging and monitoring, and a CI/CD pipeline with image scanning.